

# day18笔记

## 一、langchain版本

- 1、目前最新的是1.2.x版本
- 2、之前版本 0.1、0.2、0.3，后面就是 1.x版本
- 3、直接安装langchain，默认就是最新版本的，之前的很多的东西进行改版了，使用不了（会报错，让你采用建议的语法）
- 4、langchain查看下载版本

```
pip show langchain
```

- 5、langchain1.x以上版本，要求的 python解析器的版本最低是3.10.x及以上
- 6、由于大家的电脑系统安装的是不同的版本（专业版、旗舰版、家庭版、企业版、纯净版），执行新版本的langchain，底层需要一些解析库的支持，而你的电脑上是没有的，因此会报错。大多数问题都是你电脑系统没有C++生成解析器的库导致的

 Visual Studio 生成工具 2026 (32-bit)

- 7、你下载的如果是ARM版本的python解析器，同样的版本，x86就是没有问题，但是ARM版本就是有问题（ARM更新和优化实际上是没有那么快的，因此同样的版本的python解析器就会出问题）。建议直接安装x86版本，因为这个版本现在是支持ARM架构的

## 二、输出格式化

- 1、AI给咱们的结果，需要自定义格式让它输出，这样方便后期的使用
- 2、不使用格式化

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

template = PromptTemplate.from_template('请从下面的简介中提取姓名、年龄和技能。简介内容: {text}')
prompt = template.format(text='我叫丁致宇，今年30岁了，会python和java还有前端')

result = llm.invoke(prompt)

print(result.content)
```

结果输出

姓名：丁致宇  
年龄：30岁  
技能：Python、Java、前端

### 3、使用langchain格式化工具

#### PydanticOutputParser (老版本)

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
# pydantic 是python定义数据类型的库（类型注解）
from pydantic import BaseModel, Field
from typing import List

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 定义要输出的数据格式
# ... 表示选填选项
class PersonInfo(BaseModel):
    name: str = Field(description='人的姓名')
    age: int = Field(description='人的年龄')
    skills: List[str] = Field(description='人的技能列表')

# 初始化解析器
parser = PydanticOutputParser(pydantic_object=PersonInfo)

# template = PromptTemplate.from_template('请从下面的简介中提取姓名、年龄和技能。简介内容：{text}')
# prompt = template.format(text='我叫丁致宇，今年30岁了，会python和java还有前端')

template = ChatPromptTemplate.from_messages([
    ('system', '你是一个信息提取助手。{format_instructions}'),
    ('human', '请提取信息：{text}')
])

# 准备数据，进行解析（需要把要解析的数据给它让它进行处理）
instructions = parser.get_format_instructions()

# 把提示词模版占位的变量数据进行填充
final_message = template.format_messages(format_instructions=instructions, text='我叫丁致宇，今年30岁了，会python和java还有前端')

raw_response = llm.invoke(final_message)

# 最终变成想要的数据格式
result = parser.parse(raw_response.content)
print(result)

print('-' * 30)
```

```

# 输出json格式
print(result.model_dump_json())

print('-' * 30)

# 输出dict格式
print(result.model_dump())

# 直接提取内容
print(result.name)
print(result.age)
print(result.skills)

```

结果输出

```

name='丁致宇' age=30 skills=['python', 'java', '前端']
-----
{"name": "丁致宇", "age": 30, "skills": ["python", "java", "前端"]}
-----
{'name': '丁致宇', 'age': 30, 'skills': ['python', 'java', '前端']}
丁致宇
30
['python', 'java', '前端']

```

4、with\_structured\_output() 新版本方法

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# pydantic 是python定义数据类型的库（类型注解）
from pydantic import BaseModel, Field
from typing import List

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 定义要输出的数据格式
class PersonInfo(BaseModel):
    name: str = Field(description='人的姓名')
    age: int = Field(description='人的年龄')
    skills: List[str] = Field(description='人的技能列表')

# 直接调用模型上面提供的格式化方法，把需要处理成什么样的数据传递给这个方法处理
structured = llm.with_structured_output(PersonInfo)

result = structured.invoke('我叫丁致宇，今年30岁了，会python和java还有前端')
print(result)

print('-' * 30)

```

```
print(result.model_dump_json())
print(result.model_dump())

# print(result.name, result.age, result.skills)

d_data = result.model_dump()
print(d_data['name'])
```

### 三、大模型本地部署

- 1、模型的调用，需要花费token，因为使用的是第三方提供的API。三方公司也是有成本的
- 2、之前比较火的小龙虾（openclaw），很多人使用小龙虾的成本非常高，贵在token费用上
- 3、可以使用本地部署的方案来解决，因为现在很多优秀的模型都是开源的
- 4、部署方案，目前使用比较多的是 ollama
- 5、ollama官网地址：<https://ollama.com/>
- 6、验证ollama安装成功

```
ollama --version
```

出现版本号，那么说明就安装成功

- 7、ollama作用

- 可以让你使用这个工具下载开源模型
- 可以让你使用工具管理下载的模型（下载、删除、切换模型）

- 8、查看模型

```
ollama list
```

qwen3:1.7b 模型参数 b 表示的是10亿

- 9、下载模型

```
ollama pull qwen3:1.7b
```

```
ollama pull gemma4:31b
```

- 10、运行模型

```
ollama run qwen3:1.7b "你是谁？"
```

- 11、安装依赖

```
pip install langchain-openai
```

- 12、代码调用本地模型

## 示例代码

```
from langchain_openai import ChatOpenAI

# 注意点: api_key='ollama' 由于是本地调用模型, 不需要key, 可以填写任意字符串, 但是不能为空
llm = ChatOpenAI(
    model='qwen3:1.7b',
    base_url='http://localhost:11434/v1',
    api_key='ollama'
)

result = llm.invoke('你是谁? ')
print(result.content)
```

## 输出结果

<think>

嗯, 用户问“你是谁?”, 这可能是一个开放性的问题, 需要先确定用户的意图。首先, 我需要回应用户, 说明我的身份, 比如我是谁, 然后引导用户进一步提问, 以促进更深入的交流。

用户可能只是想了解我的基本信息, 或者他们可能对我的功能感兴趣。我需要确保回答清晰, 同时保持友好, 以便用户感到被尊重和理解。另外, 考虑到用户可能有不同的背景, 我需要用简单明了的语言, 避免使用专业术语。

同时, 我需要检查是否有潜在的问题, 比如用户是否在测试我的反应, 或者是否有其他意图。这时候, 保持开放式的提问, 如“你还有哪些问题想问我吗?”可以帮助引导用户继续对话, 进一步了解他们的需求。

总结来说, 我的回答需要包括身份说明, 功能介绍, 以及邀请用户提问, 确保回答全面且友好, 促进良好的互动体验。

</think>

我是一名AI助手, 由阿里巴巴集团研发, 专为帮助用户解答问题、提供信息和创意灵感而设计。我能够以自然的方式与您互动, 无论是日常对话还是复杂问题, 我都在努力提供准确和有用的支持。

如果您有任何问题或需要帮助, 随时告诉我! 😊

## 四、chain链

1、chain（链）是langchain核心的编排工具，本质是任务流水线

2、langchain里面有很多的核心

- 提示词（prompt）
- 模型（LLM）
- 解析器（parse）
- ...

链的作用就是把这些连接起来

3、案例对比

不适应chain

```
from langchain_community.chat_models.tongyi import ChatTongyi
```

```

from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
load_dotenv()

# 场景：实现翻译效果，输入（中文） --- 输出（英文） --- 单任务
"""
场景：多任务
+ 第一个任务：
    => 输入一段文本（可以主题不限定）
    => 把输入的文本进行总结
+ 第二个任务
    => 把第一个任务的总结，翻译成英文
+ 第三个任务
    => 把第二个任务中翻译的总结，变成一个吸引人的英文标题（提取）
"""

llm = ChatTongyi(model='qwen3-max')

# 总结：每一个任务都需要手动的进行填充，再调用模型，拿到结果。代码复用性很差
# 第一个任务，进行文本提取总结
user_text = 'Python是一种高级编程语言，语法简洁，易于学习，广泛用于AI、数据分析、web开发等领域，是目前最受欢迎语言之一'
sum_template = PromptTemplate.from_template('请总结下面的内容:{text}')
sum_prompt = sum_template.format(text=user_text)
sum_result = llm.invoke(sum_prompt).content
print(sum_result)

print('-' * 30)

# 第二个任务
trans_template = PromptTemplate.from_template('请把下面的内容翻译成英文:{text}')
trans_prompt = trans_template.format(text=sum_result)
trans_result = llm.invoke(trans_prompt).content
print(trans_result)

print('-' * 30)

# 第三个任务
title_template = PromptTemplate.from_template('请把下面的内容修改为一个吸引人英文的标题:{text}')
title_prompt = title_template.format(text=trans_result)
title_result = llm.invoke(title_prompt).content
print(title_result)

```

## 使用chain

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
load_dotenv()

# 场景：实现翻译效果，输入（中文） --- 输出（英文） --- 单任务
"""
场景：多任务
+ 第一个任务：

```

```

=> 输入一段文本（可以主题不限定）
=> 把输入的文本进行总结
+ 第二个任务
=> 把第一个任务的总结，翻译成英文
+ 第三个任务
=> 把第二个任务中翻译的总结，变成一个吸引人的英文标题（提取）
"""

llm = ChatTongyi(model='qwen3-max')

# 用户输入的文本信息
user_text = 'Python是一种高级编程语言，语法简洁，易于学习，广泛用于AI、数据分析、web开发等领域，是目前最受欢迎语言之一'

# 第一个任务，提示词
sum_template = PromptTemplate.from_template('请总结下面的内容:{text}')

# 第二个任务，提示词
trans_template = PromptTemplate.from_template('请把下面的内容翻译成英文:{text}')

# 第三个任务，提示词
title_template = PromptTemplate.from_template('请把下面的内容修改为一个吸引人英文的标题:{text}')

# 使用chain（链）
# | 管道符
chain = sum_template | llm | trans_template | llm | title_template | llm

# 一键调用
final_result = chain.invoke({'text': user_text})

print(final_result.content)

```

## 五、RunnableParallel

1、RunnableParallel是链中的一个方法，可以同时运行多个任务

2、示例代码

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 用户输入的文本信息
user_text = 'Python是一种高级编程语言，语法简洁，易于学习，广泛用于AI、数据分析、web开发等领域，是目前最受欢迎语言之一'

# 第一个任务，提示词
sum_template = PromptTemplate.from_template('请总结下面的内容:{text}')

# 第二个任务，提示词
trans_template = PromptTemplate.from_template('请把下面的内容翻译成英文:{text}')

```

```

# 第三个任务，提示词
title_template = PromptTemplate.from_template('请把下面的内容修改为一个吸引人英文的标题:{text}')

# 同时运行多个任务并拿到每一个任务的结果
from langchain_core.runnables import RunnableParallel

# 使用chain
# 参数要求是一个字典，因为要执行多任务
chain = RunnableParallel({
    'sum_result': sum_template | llm,
    'trans_result': trans_template | llm,
    'title_result': title_template | llm
})

# 拿到结果
result = chain.invoke({'text': user_text})

# 拿到第一任务的结果
print('总结:', result['sum_result'].content)
# 拿到第二任务的结果
print('翻译:', result['trans_result'].content)
# 拿到第三任务的结果
print('标题:', result['title_result'].content)

```

## 六、LLMchain基础链

- 1、LLMchain是链中最简单的一种的，也是其他链的基石，其他链都是对它的扩展
- 2、需求：根据“主题、受众、字数”三个参数，生成对应的科普短文。

不使用基础链

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 需求：根据“主题、受众、字数”三个参数，生成对应的科普短文。
# audience 受众（小学生、中学生、大学生）
# word_count 字数
# topic 主题
template = PromptTemplate.from_template('给{audience}写一篇{word_count}字左右的{topic}科普短文，语言通俗，避免专业术语，加入简单例子。')
prompt = template.format(audience='中学生', word_count=200, topic='人工智能')

result = llm.invoke(prompt).content
print(result)

```

使用基础链（代码灵活性更强、可复用、一定程度上减少了代码量）



```
# 使用基础链
llm = ChatTongyi(model='qwen3-max')

prompt = ChatPromptTemplate.from_messages([
    ('human', '给{audience}写一篇{word_count}字左右的{topic}科普短文，语言通俗，避免专业术语，加入简单例子。')
])

# llm 表示使用那个模型
# prompt 表示提示词
# output_key 定义最终输出结果的字段，之前很多默认的都是 content
# verbose=True 可以看到模型执行过程，可以理解为日志
chain = LLMChain(
    llm=llm,
    prompt=prompt,
    output_key='科普短文',
    verbose=True
)

result1 = chain.invoke({
    'audience': '小学生',
    'word_count': 200,
    'topic': '人工智能'
})

print(result1['科普短文'])

print('-' * 50)

result2 = chain.invoke({
    'audience': '中学生',
    'word_count': 300,
    'topic': 'AI'
})

print(result2['科普短文'])

print('-' * 50)

result3 = chain.invoke({
    'audience': '大学生',
    'word_count': 200,
    'topic': '数据分析'
})

print(result3['科普短文'])
```

## 七、作业：

- 1、**文章生成器**，结合streamlit，结合链去做
- 2、翻译助手（多国语言翻译），结合streamlit，结合链去做